LogiEdge Phase 2 Report

Training, normalisation experiment, deployment, MLOps, and model choice

AIML ZG535 — Machine Learning on Edge · Continuous Evaluation Component EC-I
Birla Institute of Technology & Science, Pilani · Work Integrated Learning Programmes Division

Modules 3–6 (measured). Constraint analysis, the hardware choice and the MQTT design are in the Phase 1 report. The edge node is the Raspberry Pi 5. This report uses the laptop measurements already in the repository.

1. Dataset and training

The simulator publishes temperature, vibration RMS and door events to the local broker. --anomaly selects none, temp_drift (+0.08 °C per reading), vibration (step to about 1.2 g), or combined. Windows are 30 seconds long and step every 10 seconds, after a five-sample moving average. The committed files hold 117 Normal, 87 Warning and 87 Critical windows, close to the requested 20, 15 and 15 minutes.

The classifier is a multilayer perceptron with hidden widths 32 and 16, ReLU, and a three-way softmax. The saved run fitted 235 windows. A stratified 20 percent holdout of 59 windows (24 Normal, 18 Warning, 17 Critical) is shared by every variant. The script refuses to save a model below 88 percent validation accuracy or 95 percent Critical recall. The FP32 model scores 100 percent accuracy and 17/17 Critical recall.

2. Normalisation experiment

This is the mandatory Phase 2 experiment. Mean and standard deviation of the six features were taken from clean Normal output and stored in training_stats.npy. At runtime the service loads that file. It does not recompute statistics from the live stream. Several standard deviations are floored, 0.1 on the tightly clustered Normal features, 0.2 on the temperature rate of change and 1.0 on kurtosis, and the normalised vector is clipped to ±10.

The same 59-window holdout was then scored with the stored mean shifted by three of those standard deviations. A +3σ shift moves every normalised feature down by 3. A -3σ shift moves every normalised feature up by 3. No weights were retrained.

Statistics	M1 accuracy	M2 accuracy	M3 accuracy	What changed
Correct file	100%	100%	100%	Critical recall 100% on all three (17/17)
Mean +3σ	98.3%	100%	98.3%	M1 misses 1 Critical (recall 94.1%). M3 calls 1 Warning Normal. Critical recall on M3 stays 100%
Mean -3σ	61.0%	61.0%	61.0%	23 of 24 Normal windows are called Critical. Warning and Critical recall stay 100%

With the correct file the confusion matrix is diagonal: 24, 18 and 17. The +3σ shift is small and uneven. M2 is unchanged. M1 calls one Critical window Normal, so Critical recall falls to 94.1 percent, under the 95 percent gate. M3 calls one Warning window Normal and still catches every Critical window. The -3σ shift is a false-alarm failure on every variant. Normal recall falls to 1/24 (4.2 percent). A healthy compartment looks Critical because the model was told that "normal" sits three standard deviations away from the values it was trained on.

The statistics file is therefore part of the release, not a notebook leftover. The deployment playbook copies `reference_dist.json` and the model together, and the same rule applies to training_stats.npy: a truck must not rebuild the scale from whatever cargo it happens to be carrying. Phase 1's bandwidth argument only holds if this file is right. A bad scale creates a fleet of false Critical alerts, which is its own uplink cost and a trust problem with the hospital customers.

3. Ten-stage pipeline, Docker, and OTA

LogiEdge maps onto the ten edge-pipeline stages as follows:

- **Problem framing** fixes three compartment classes, a 90-second alert, and no dependence on continuous cellular coverage.
- **Data acquisition** is the simulator on the local broker, with the four anomaly modes above.
- **Preprocessing** is the five-sample moving average.
- **Features and fusion** build the six-value vector and scale it with the frozen statistics.
- **Training** fits the 32-by-16 perceptron on the generated windows.
- **Evaluation** is the shared 59-window holdout, gated at 88 percent accuracy and 95 percent Critical recall.

- **Optimisation** builds M2 by full-integer post-training quantisation and M3 by removing about 35 percent of the hidden neurons before that quantisation.
- **Conversion** writes TensorFlow Lite with integer weights, activations and an integer input quantiser, calibrated on the training windows (235 samples, above the 200-sample minimum).
- **Packaging** is the container and the Ansible copy onto the node.
- **Monitoring** is the population-stability sidecar, shipped with its reference histogram.

The image is `python:3.11-slim`. Every `pip install`, including TensorFlow, is built before the copy of the model files, and that copy is the last filesystem layer. MODEL_PATH selects the file at start, so switching among the FP32 file, plain INT8 and pruned INT8 does not require a rebuild. A model-only rebuild re-runs the last layer and leaves the runtime cached on the truck. The three files in that layer are 34.46 KB, 4.58 KB and 3.75 KB, 43,819 bytes together. At €0.10 per MB (1 MB = 10^6 bytes) one truck downloads 0.044 MB and 85 trucks download 3.72 MB, which costs €0.37. Pulling the whole image again would repeat the TensorFlow layer on every truck. The inference service publishes to logibridge/trucks/{truck_id}/inference.

FreightBridge plans a weight update every six weeks. The brief prices that file at 280 KB, which is 0.2734 MB, on an M2M SIM at €0.10/MB. A cycle that reaches all 85 trucks moves 23.24 MB and costs €2.32 whichever strategy is used. The trained pruned file is 3.75 KB, about 75 times smaller, so €2.32 is a conservative cap for the planning figure in the brief.

Strategy	First wave	Rest of the fleet	Cost if the cycle completes
Full replacement	85 trucks, €2.32	—	23.24 MB, €2.32
Canary	10 trucks, €0.27	75 trucks, €2.05	23.24 MB, €2.32
Shadow mode	10 trucks, €0.27	75 trucks, €2.05	23.24 MB, €2.32

Full replacement is the simplest operation and the worst fault. A bad calibration would sit on every pharmaceutical truck at once, including trucks inside a 35–90 minute coverage gap that cannot roll back until they next see a tower. Shadow mode keeps the current model in control while the new one runs beside it. That is the strongest check, and it costs a second inference plus a comparison that may see no Critical event in six rural weeks. Silence then looks like agreement. Canary deployment is the recommendation. Ten trucks take the file first. If their drift monitor and Critical detections stay healthy, the other 75 follow. The completed cycle costs the same €2.32, the blast radius of a bad file is ten trucks, and a failed canary is reversed by re-running the playbook with the previous model path.

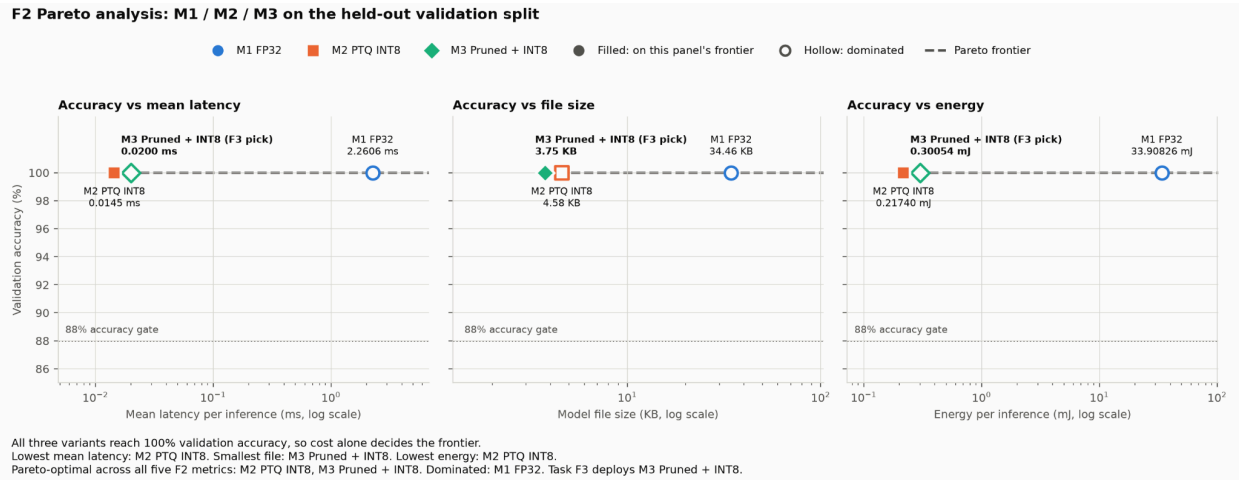
4. Benchmark performance and deployment decision

Process CPU during the runs was about 5–8 percent and is not multiplied in. Pi 5 times below assume a pessimistic tenfold slowdown against this laptop. They should be replaced by one on-device run of the same script before the pilot exits.

Variant	Mean latency (ms)	p95 latency (ms)	Size (KB)	Accuracy (%)	Energy (mJ)
M1 FP32	2.261	3.443	34.46	100	33.91
M2 PTQ INT8	0.0145	0.0149	4.58	100	0.217
M3 pruned + INT8	0.0200	0.0359	3.75	100	0.301

Table. Fifteen measurements on the 59-window holdout. Critical recall is 100 percent (17/17) on every row. M1 called through `predict`, as the service does, is 63.9 ms mean and 73.6 ms at p95. The M1 SavedModel export is 51.43 KB.

Accuracy does not separate the three models. INT8 matches FP32 on every validation window. Latency and energy put M1 more than a hundred times behind the integer models, and they leave M2 and M3 tied: across repeats those means sit between 0.010 and 0.022 ms and swap which one is faster. Size does not swap. M3 is 18 percent smaller on disk than M2 and uses 0.53 KB of weights and activations against 0.99 KB. All three points sit at 100 percent, above the 88 percent gate. M1 is inside the frontier on every panel. M2 and M3 are both on it. The tie-break that survives the next benchmark run is file size, so M3 is the deployed model and M2 stays in the image as the rollback.



The 90-second SLA leaves about 40 seconds after windowing (10 seconds to flush the vibration average, 30 seconds to fill the window, up to 10 seconds until the next step). The service evaluates every 10 seconds on one thread, so a single inference slower than 10 seconds would push the next alert past the SLA. The engineering target is 100 ms on the device. M3's laptop p95 is 0.036 ms, about 0.4 ms under the tenfold Pi 5 assumption, which is under half a percent of 100 ms. M1 through predict, scaled the same way, is the only variant that threatens that target. On the Pi 5, M3's weights are 0.10 percent of the 512 KB L2 cache per core, and the file is negligible on the microSD card. Both integer models need only the TFLite interpreter.

Critical recall on M3 is 17/17, above the 95 percent gate, and the conversion script fails if it is not. One miss on this holdout would drop recall to 94.1 percent. Seventeen clean detections support a 95 percent lower confidence bound near 84 percent, so the pilot exit criterion is at least 59 confirmed Critical events with none missed. There is a second limit in the labels. Warning windows come from temperature drift and Critical windows from that drift plus the vibration step, so temperature alone does not define the class. After pruning, M3 follows a rising temperature more than a high one. The integer input range maps temperature mean only from about 3.74 °C to 5.01 °C, so a cold excursion is invisible to M2 and M3.

The service therefore applies a rule after the model: a window mean more than 1 °C from the 4 °C setpoint is at least Warning, and more than 3 °C away is Critical, on either side. The rule only raises the model's class, so a vibration fault is still caught by M3. Combined Critical recall on the holdout stays 17/17.

Deploy M3 (cold_chain_mlp_pruned_int8.tflite) on the Pi 5 CPU, with that temperature rule in the same container. Keep M2 as the rollback. A rollback is a second playbook run with the M2 path, or a MODEL_PATH change, and does not need a new image. M1 is rejected on `predict` latency, on file size, and on the full TensorFlow runtime it would put on every truck.

5. Drift monitoring and Ansible

Drift is watched on the model's probability of Normal, the first entry of probabilities on the local inference topic. The reference is a four-bin histogram of that score on 300 clean windows, with edges at 0, 0.25, 0.50, 0.75 and 1. The sidecar keeps the last 100 scores, recomputes the population stability index every 60 seconds, and prints `[LOGIBRIDGE DRIFT ALERT]` when the index exceeds 0.25. Empty bins take a Jeffreys offset of one half, so an unseen bin cannot send the index to infinity.

Using that function, 300 reference scores in the top bin and a clean window of 100 scores in the same bin give an index of 0.011. That is the value before injection while the model is confidently Normal. When the combined anomaly fills the window with scores in the bottom bin, the index rises to 11.55. It crosses 0.25 as soon as a minority of the 100 scores leave the top bin, which is inside

the five-minute limit at one inference every 10 seconds. Restoring clean data returns the index to 0.011 once the rolling window has flushed. The recovery line is 0.10.

The injected fault is a sudden covariate shift: a temperature ramp plus a vibration step. That is data drift, not a change in what the labels mean, and it shows up as prediction drift because P(Normal) collapses when the model commits to Critical. Watching the maximum class probability would miss the event. Under the combined fault the model is confidently Critical, the maximum stays in the top bin, and the index stays near 0.011. The score under test has to be the probability of the healthy class.

deployment/logibridge_deploy.yml has the seven required tasks: create /opt/logibridge, copy the model to `model.tflite`, copy `reference_dist.json`, stop the container only if a copy changed, pull the image from the local registry, start it with MODEL_PATH, the truck id and the broker, then wait 15 seconds and accept it only if it is running and the restart count is zero. The default file is M3. A second run with nothing changed between the two invocations must report changed=0. Copies are checksummed, the stop is skipped when both files match, and the health check does not count as a change. The continuous-integration job runs the play twice and fails the build if the second host line is not changed=0 with failed=0.